

MODULE 30 · DOMAIN 3 - CONTINUOUS IMPROVEMENT FOR EXISTING SOLUTIONS

Improving Reliability

Find what fails, then ask what else fails with it.

DOMAIN 3 of 4 EXAM WEIGHT 25% MODULE 30 of 44

Recovery that needs AWS to **create** something during an outage may be waiting in the same queue as everyone else.

Control plane and data plane

- ◆ **The control plane creates and changes things:** launching an instance, creating a bucket, modifying a route table.
- ◆ **The data plane uses what exists:** serving a request, reading an object, routing a packet already provisioned for.
- ◆ **Data planes are designed to be more available than control planes.** That asymmetry is deliberate and documented.
- ◆ So a recovery path that depends on control plane actions is less reliable than the thing it is recovering.

ON THE EXAM

AWS's own example: in Kubernetes, adding pods is a data plane action and adding nodes is control plane. Failover should add pods.

Static stability: the system keeps working during a failure **without needing anything new to be created.**

Self-healing, and what it does not cover

- ◆ **Simplified automatic recovery is on by default**, moving an instance off a failing host when a system status check fails.
- ◆ **It is not initiated for instances in an Auto Scaling group**. The group replaces the instance instead.
- ◆ **It needs capacity to be available**, so during a large outage recovery attempts can fail. AWS says so plainly.
- ◆ ASG health checks, ELB health checks and Route 53 health checks each remove a failed component at a different layer.

ON THE EXAM

Automatic recovery is a control plane action too. It works well for one failed host and less well when a whole zone is short of capacity.

Limiting blast radius

- ◆ **Cell-based architecture** runs many independent copies of the stack, each serving a subset of customers. A failure takes out one cell.
- ◆ **Shuffle sharding** assigns each customer a different combination of cells, so no two customers share the same full set.
- ◆ **Bulkheads** separate resources so one workload cannot exhaust what another needs, such as separate connection pools.
- ◆ The cost is operational. Twenty cells means twenty deployments, twenty sets of metrics, and a routing layer that must never fail.

ON THE EXAM

Shuffle sharding is the elegant one. With eight cells and two per customer, a bad tenant affects a small fraction rather than everyone.

Client-side reliability

- ◆ **Naive retries amplify an outage.** A struggling service receiving three times the traffic does not recover.
- ◆ **Exponential backoff with jitter** spreads retries out. Without jitter, every client retries at the same moment.
- ◆ **Circuit breakers** stop calling a failing dependency entirely for a period, so it gets space to recover.
- ◆ **Timeouts must be shorter than the caller's timeout**, or work continues on a request nobody is waiting for.

ON THE EXAM

Jitter is the part people omit. Backoff alone synchronises every client onto the same retry schedule, which is a thundering herd.

Testing that it works

- ◆ **A game day exercises a failure deliberately**, in a controlled window, with the team that would respond.
- ◆ **Start with a dependency map.** You cannot test failure modes you have not identified.
- ◆ Chaos engineering is the discipline of injecting failure continuously rather than annually.
- ◆ **Scope note:** Fault Injection Service, Resilience Hub and Application Recovery Controller are not on the exam's in-scope list.

ON THE EXAM

Module nineteen said an untested plan is a hypothesis. The same is true of a resilience design that has never lost anything on purpose.

Reading the scenario

THE SCENARIO SAYS

Failover depends on launching instances

One customer's load affects everyone

Retries made the outage worse

A single NAT gateway or single AZ component

The instance did not auto-recover

Failover has never actually been tested

IT IS POINTING AT

Static stability: pre-provision capacity

Cell-based architecture, shuffle sharding

Exponential backoff with jitter

A single point of failure

It is in an Auto Scaling group

Run a game day

ON THE EXAM

Ask what has to happen during the failure. If the answer includes creating something, that is the weakness.

Meridian Freight runs an Auto Scaling group across three Availability Zones, sized so all three together exactly meet peak demand. The plan for an AZ failure is that Auto Scaling launches replacement instances in the surviving zones.

During a recent AZ event, those launches were throttled and the service was degraded for 50 minutes.

What should the solutions architect recommend?

- ☐ A Pre-provision capacity so any two zones can serve full demand, making the design statically stable
- ☐ B Increase the Auto Scaling group's maximum size
- ☐ C Add a fourth Availability Zone to the group
- ☐ D Reduce the health check grace period so replacements start sooner

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A** Pre-provision for N-1
Nothing needs creating when a zone fails, so the control plane is not on the recovery path at all. That is static stability.
- B** Raise the maximum
The group was not at its ceiling. It was throttled, which is a different failure, and a higher maximum does not get you capacity.
- C** A fourth AZ
Reduces the share lost to one zone, and still requires launching instances during the event. The dependency remains.
- D** Shorter grace period
Speeds up instances that did launch. The problem was launches being throttled, not being slow to become healthy.

THE SIGNAL

Throttled during an AZ event is the giveaway. Every customer's recovery plan fired at once, and yours was in the queue.

An operations team reports that a standalone EC2 instance recovered automatically when its host failed, but an identical instance inside an Auto Scaling group did not, and stayed impaired for 20 minutes until someone noticed.

The Auto Scaling group uses EC2 status checks only.

What is the explanation, and what should change?

- ☐ A Automatic recovery is not initiated inside an Auto Scaling group; enable ELB health checks so the group replaces impaired instances
- ☐ B Automatic recovery must be enabled manually on instances in a group
- ☐ C The instance type does not support automatic recovery
- ☐ D The Auto Scaling group needs a lifecycle hook

◆ PAUSE AND DECIDE

What is the explanation, and what should change?

- A

 ASG replaces instead; use ELB health checks
Documented behaviour: auto recovery is not initiated inside an ASG. Replacement is the group's job, and ELB health checks catch application failures too.
- B

 Enable it manually
It is on by default on supported types. The group's presence is what suppresses it, not a missing setting.
- C

 Unsupported type
The scenario says an identical instance outside the group recovered. Same type, different outcome.
- D

 Lifecycle hook
Hooks pause instances entering or leaving service. They do not detect or replace an impaired instance.

THE SIGNAL

Identical instance, different result, one variable: membership of an Auto Scaling group.
That is the controlled experiment again.

A SaaS platform serves 900 customers from one shared stack. Twice this year, a single customer's unusual query pattern exhausted the database connection pool and every customer experienced an outage.

The business wants one customer's behaviour to affect as few others as possible, without running 900 copies of the stack.

What should the solutions architect recommend?

- ☐ A A cell-based architecture with shuffle sharding, assigning each customer a different combination of cells
- ☐ B Rate limiting per customer at the API layer
- ☐ C A larger database instance with a bigger connection pool
- ☐ D A dedicated stack for each of the 900 customers

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

 Cells with shuffle sharding
A handful of cells, each customer on a different combination, so one bad tenant affects a small overlapping fraction rather than everyone.
- B

 Per-customer rate limiting
Worth doing and it limits request volume, not resource consumption. A small number of expensive queries still exhausts the pool.
- C

 Bigger database
Raises the ceiling that was hit twice. The next unusual query pattern finds the new ceiling.
- D

 900 stacks
Perfect isolation and explicitly ruled out by the scenario. It is also 900 deployments to operate.

THE SIGNAL

Without running 900 copies is the planted constraint. Shuffle sharding gets most of the isolation from a handful of cells.

A downstream service became slow for 90 seconds. Clients retried immediately, three times each. The retry traffic tripled load on the already-struggling service, which then failed completely and stayed down for 40 minutes.

Every client is configured to retry three times with no delay.

What should the solutions architect recommend?

- ☐ A Exponential backoff with jitter, plus a circuit breaker that stops calling the dependency after repeated failures
- ☐ B Increase retries from three to five so requests eventually succeed
- ☐ C Scale the downstream service to handle the retry load
- ☐ D Remove retries entirely

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A** Backoff with jitter, plus a breaker
Backoff spreads retries over time, jitter stops them synchronising, and the breaker gives the dependency space to recover.
- B** More retries
Amplifies the amplification. Five retries each is five times load on a service that failed under three.
- C** Scale for retry load
Sizes the system for its own failure mode, permanently, so you pay every hour for capacity that exists only for retry storms.
- D** No retries
Removes the amplification and also the resilience. A transient blip now fails every request that hits it.

THE SIGNAL

Ninety seconds of slowness became forty minutes of outage. That gap is the retry storm, not the original fault.

A resilience review of a three-tier application finds: instances across three AZs, an RDS Multi-AZ database, an Application Load Balancer, and one NAT gateway in a single AZ serving all private subnets. All outbound traffic to a payment provider goes through it. Everything else is considered redundant.

What is the most significant single point of failure?

- ☐ A The single NAT gateway, which fails with its Availability Zone
- ☐ B The Application Load Balancer
- ☐ C The RDS Multi-AZ database
- ☐ D There is no single point of failure in this design

◆ PAUSE AND DECIDE

What is the most significant single point of failure?

- A

The single NAT gateway

A NAT gateway is zonal. Losing that AZ removes outbound connectivity for every private subnet, so payments fail from all three tiers.
- B

The load balancer

An ALB is a regional service with nodes in each enabled subnet. It is not a single point of failure by design.
- C

The database

Multi-AZ provides a standby with automatic failover. It is one of the redundant parts here.
- D

No SPOF

The review's own conclusion, and it missed the NAT gateway because it is easy to read as shared infrastructure rather than zonal.

THE SIGNAL

NAT gateways are zonal, not regional. One per AZ with per-AZ route tables is the fix, and the extra cost is the trade.

A company has invested heavily in resilience: multi-AZ, cross-Region replication, health checks and documented runbooks. None of it has ever been exercised. Leadership asks for assurance before an upcoming regulatory review.

The team is nervous about testing in production.

What should the solutions architect recommend?

- A Run a scheduled game day in a controlled window, starting with a dependency map and a limited failure, with the on-call team responding
- B Produce architecture documentation demonstrating the redundancy
- C Test only in the staging environment, permanently
- D Assume the design works, since each component is individually redundant

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

A controlled game day

Produces evidence rather than assertion, and a controlled window with a limited failure addresses the team's nervousness directly.
- B

Documentation

Describes the intended behaviour. A regulator asking for assurance is asking what happened when you tried, not what you drew.
- C

Staging only, forever

Useful and never conclusive. Staging differs in scale, data, traffic and configuration, which is where failures hide.
- D

Assume it works

Individually redundant components frequently share a dependency nobody mapped. That is what a game day finds.

THE SIGNAL

Module nineteen said an untested DR plan is a hypothesis. A resilience design that has never lost anything on purpose is the same thing.

What to carry out of this module

- ◆ **Static stability:** if recovery requires creating something, it depends on the control plane at the worst moment.
- ◆ **Automatic recovery is not initiated inside an Auto Scaling group**, and it needs available capacity to work at all.
- ◆ **Shuffle sharding** buys most of the isolation of per-customer stacks from a handful of cells.
- ◆ **Jitter, not just backoff.** Backoff alone synchronises every client onto the same retry moment.

ON THE EXAM

NAT gateways are zonal. Three AZs of compute routing out through one gateway is one AZ of availability.